

Smart Ambulance Routing & Emergency Bed Allocation

Team explainer + full judge pitch content

*"We do not simply navigate ambulances.
We coordinate the entire emergency journey."*

PROJECT AT A GLANCE

What it is	A coordination platform linking ambulance selection, routing, hospital matching, acceptance, and bed reservation into one live decision
Status	Working prototype — full incident-to-arrival pipeline built and tested through build step S15 (one step, S13 ML/voice, deliberately skipped — see below)
Verified	33 backend tests, 6 frontend tests, 7 Playwright browser tests, and one live end-to-end run all passed
Honest limits	Hospital capacity data is simulated and labelled; ML/voice deliberately skipped rather than shipping unverified accuracy claims

CONTENTS

Part 1 — Team-member explainer

Part 2 — Judge pitch:

1. The problem
2. Our idea (overview)
3. How it works
4. Tech stack
5. Feasibility — what works today
6. Implementation process
7. Practicality — making this real
8. Why it matters

Part 1 — Team-Member Explainer

Short, beginner-friendly. For a teammate who hasn't read the specs.

Right now, the ambulance, the route, and the hospital for an emergency get picked almost separately — so a patient can end up at a hospital with no bed ready for them. Our project connects all three into one system: it picks the right ambulance, the fastest route, and a hospital that has actually agreed to take the patient and has a bed held and waiting. If anything changes on the way — traffic, a lost bed, a broken-down ambulance — it automatically re-checks everything, live. The hospital data in our demo is simulated and clearly labelled as such; everything else is real and working end to end.

Part 2 — Judge Pitch

Full spoken content, in presentation order. Beginner-friendly, persuasive, and honest about limits.

1. The Problem

Every year, precious minutes are lost in emergencies — not because ambulances are slow, but because of something less visible: **bad coordination**. An ambulance can be sent that isn't equipped for the patient's actual condition. It can be routed to the nearest hospital, only for that hospital to have no ICU bed free, or to reject the patient on arrival — forcing a second, more dangerous journey while the clock keeps running.

Today, four decisions — which ambulance, which route, which hospital, and whether a bed is actually ready — are handled as separate, disconnected steps. Our project treats them as **one decision, made together, and kept up to date as things change**.

2. Our Idea (Overview)

The Smart Ambulance Routing & Emergency Bed Allocation System is a coordination platform that manages the entire emergency journey — from the moment a call comes in, to a hospital bed being physically ready for the patient.

Given an emergency and the patient's needs, the system doesn't just find the nearest ambulance and nearest hospital. It:

- finds the ambulance that is actually free *and* equipped for this specific patient;
- calculates the fastest reliable route, not just the shortest one;
- filters hospitals down to the ones that can medically handle the case;
- ranks those hospitals by real capability and likelihood of acceptance;
- gets that hospital to explicitly say yes; and
- reserves the exact resource — an ICU bed, a ventilator, a trauma bay — before the ambulance ever arrives.

Nearest is not necessarily fastest.

Fastest is not necessarily suitable.

Suitable is not necessarily accepting.

Available now is not necessarily available on arrival.

We built the whole system to solve for the **best feasible chain** — not the closest single point.

3. How It Works

Emergency reported → patient needs captured (e.g. "ICU required", "trauma required") → ambulances scored on suitability, availability & ETA → best ambulance assigned → real route calculated, with alternatives considered → hospitals filtered by medical capability and current resource state → hospitals ranked → hospital staff explicitly accept or reject on their own dashboard → on acceptance, the specific resource is reserved — server-side, transaction-locked, so two ambulances can never claim the same bed → the mission is tracked live over a real-time connection → if conditions change, the system automatically reassesses and can re-route or re-rank → the hospital gets advance notice to prepare → patient arrives at a hospital that is ready and waiting.

Two things make this trustworthy rather than a black box: every recommendation comes with a plain-language **explanation** of why that ambulance or hospital was chosen, and a human dispatcher can **override** any decision at any time. The system never invents a recommendation when nothing genuinely qualifies — it says so instead.

4. Tech Stack

Frontend — what dispatchers, hospitals and crews actually see

React + TypeScript

Vite

Tailwind CSS

TanStack Query

Zustand

React Router

React Hook Form + Zod

Recharts

Three live screens — dispatcher, hospital, and ambulance crew — plus a demo controller, built quickly and kept type-safe with React/TypeScript on Vite. TanStack Query and Zustand keep every screen automatically in sync with the live backend instead of needing manual refreshes; React Hook Form and Zod validate inputs (like patient requirements) before they ever reach the server.

Backend — the decision-making brain

Python

FastAPI

Pydantic

SQLAlchemy

Alembic

WebSockets

FastAPI runs all the matching, ranking, and business rules, with Pydantic validating every request. SQLAlchemy and Alembic manage the database schema safely as it evolves. WebSockets push live updates — a hospital accepting, a bed being reserved, a route changing — to every open screen instantly.

Database — where location actually matters

PostgreSQL

PostGIS

PostGIS is a real spatial database extension, so "which ambulance/hospital is actually near this incident" is a genuine geographic calculation, not a rough approximation.

Optimization & AI — used only where they earn their place

Google OR-Tools

XGBoost / LightGBM

scikit-learn

OR-Tools is available for the matching layer where a proper solver beats simple sorting. XGBoost/scikit-learn power an optional arrival-time prediction layer — used only where a prediction genuinely improves a decision, and never used to claim an accuracy number we haven't actually measured.

Testing — proving it works, not just claiming it

Pytest

Vitest

React Testing Library

Playwright

Backend logic, frontend components, and full real-browser flows are all covered by automated tests (results in the Feasibility section below).

5. Feasibility — What Actually Works Today

This is not a mockup or a slide deck pretending to be a product. As of today, the build covers every planned step through migration 0015_simulation_integrity: authentication and role-based access, the full incident → ambulance → route → hospital → acceptance → reservation pipeline, live recalculation when conditions change, failure simulation, and authenticated real-time updates over WebSockets — across all three live dashboards.

Verification	Result
Backend unit/integration tests	33 passed; linting and compilation clean
Database integrity tests	Concurrent reservation, rollback, and migration round-trip all passed — double-booking a bed is actually prevented at the database level, not just in theory
Frontend tests	Build, lint, and 6 Vitest tests passed
Browser tests (mocked)	6 accessibility & dispatcher-flow Playwright tests passed
Browser test (live stack)	1 full Playwright run passed against the real running FastAPI + PostgreSQL system
Live end-to-end walkthrough	Login → incident → requirements → ambulance match → confirmation → hospital match → route calculation, all executed live
Matcher latency	50 deterministic samples: p50 0.009 ms, p95 0.012 ms (matching-engine latency, not full network/browser latency)

Said plainly, up front

Hospital capacity and acceptance are explicitly **simulated** in this prototype and labelled "DEMO MODE" in the UI — we are not presenting simulated data as live. The routing provider is a deterministic mock, which is what makes the demo reliable and able to run offline. The optional

ML/voice layer (S13) was intentionally skipped rather than shipped with a fabricated accuracy number, because the project has a hard rule against inventing numbers.

6. Implementation Process

The system was built in a fixed phase order, and each phase had to produce a genuinely working output before the next one started — nothing was designed in isolation and stitched together at the end:

Phase	What it delivered
Foundation	Repository, auth, base UI — a running application
Core data model	Incidents, ambulances, hospitals, resources, reservations, missions
Ambulance matching	Capability + availability + ETA scoring → working ambulance selection
Routing	Real route calculation with alternatives → working map
Hospital matching	Capability + resource filtering → ranked, explained recommendation
Acceptance	Hospital dashboard with accept/reject/timeout → confirmed destination
Reservation	Transactional hold/confirm/release/expiry → safe resource reservation
Real-time coordination	WebSockets, live state, dynamic re-ranking → a truly live mission
Failure simulation	Traffic change, ICU loss, hospital rejection, ambulance/GPS failure → stress-tested live
AI layer (optional)	ETA prediction with confidence, added only after everything else worked
QA & demo rehearsal	Full scenario validation and presentation polish

This order was deliberate: get one complete, real, end-to-end emergency working first, then layer on real-time behaviour, failure handling, and AI — never the reverse.

7. Practicality — Making This Real

The architecture was designed for the handoff to production from day one, not bolted on afterward:

- **A clean integration seam is already built in.** Hospital data sits behind an integration interface. Moving from simulated hospital capacity to a real, authenticated hospital feed is a matter of plugging in an adapter — not redesigning the core system.
- **The security and audit basics real partners ask for first are already in place** — role-based access control, authenticated real-time connections, and an audit trail of every critical

decision.

- **Double-booking is already impossible**, because reservations are server-side, transaction-locked, and database-enforced — this is the piece hospitals and regulators would scrutinize hardest, and it's proven, not promised.
- **A realistic rollout path**: pilot in a single city or district with a small set of partner hospitals; swap their data from simulated to a live feed through the existing adapter once an integration agreement is signed; expand ambulance and hospital coverage gradually from there.
- **The production evolution is mapped, not hand-waved** — from today's single backend service to dedicated dispatch, hospital, routing, ML and reservation services connected to real hospital systems, ambulance tracking (AVL), traffic providers, and emergency-response infrastructure.

What this prototype cannot yet validate

Real clinical outcomes, actual mortality impact, live nationwide hospital integration, hospital cooperation at scale, and performance under real emergency-traffic load. We say this plainly rather than implying it's already solved — the roadmap above is how we'd close each gap.

8. Why It Matters

Every part of this system exists because emergencies are won or lost on minutes, and today those minutes get wasted on decisions made separately: the right ambulance sent to the wrong hospital, or the right hospital found too late to get ready. We built the coordination layer that closes that gap.

"We're not just moving an ambulance. We're getting a patient to a hospital bed that is medically right, has already said yes, and is already waiting for them."